從第一原理理解 YuniKorn

從「誰該拿到資源？」到第一次讀懂 YuniKorn Issue

Version 1.0 · Short Main Path

Prototype for Review

TECHNICAL BASELINE
Apache YuniKorn 1.9.0

MODE
standard scheduler mode

AUTHOR
Yen-Hua Chen

LICENSE
CC BY 4.0

LANGUAGE
Traditional Chinese · zh-TW

READING TIME
2–3 hours

Repository source
https://github.com/hikaru-212/open-source-from-first-principles

這是一份教學原型，不是完整的 YuniKorn 規格、Kubernetes scheduler 教科書或設定手冊。若本文與其他資料有差異，應以 Apache YuniKorn 1.9.0 官方文件，以及本文固定版本的 core、Kubernetes shim 原始碼與 stable tests 為準。

YuniKorn 到底是什麼？

YuniKorn 是 Kubernetes 上的多租戶資源排程器（multi-tenant resource scheduler）。

當多個 applications / queues 競爭有限的 CPU、memory 與其他叢集容量時，它依政策決定誰先被考慮、哪筆資源需求可獲得 allocation、應放到哪個可行的 node，以及必要時哪些既有 allocations 可被 preempt。它處理的是「有限資源應該分給誰」；被優先考慮不代表一定放得下，選定搶占對象也不代表資源已釋放。

YuniKorn 不負責真正建立 container 或執行 process，也不能單靠 scheduler decision 證明 workload 已開始有用的計算。

有了這個定位，第一章再追問：到底在分配什麼？

接下來，這份指南追問三件事：

> YuniKorn 為什麼存在？它有權決定什麼？從資源需求到真正進展之間，哪裡可能失敗？

怎麼讀這份指南

第一次閱讀時，不需要記住所有 internal 名稱。每一章只追蹤四個問題：

1. 現在是誰在要求資源？
2. 哪個 component 有權決定什麼？
3. scheduler state 與 Kubernetes / runtime state 哪裡可能不同？
4. failure 後，哪個 transition 還沒完成？

`Deep Dive`、精確 source path 與 issue number 都可以留到第二遍再看。

第一章：到底在分配什麼？

假設一個資料處理工作建立三個 Pod。三者屬於同一份工作，但各有自己的 Pod UID，也各自宣告不同的 CPU 與 memory request：

```
Application analytics-job
  ├── Pod UID p-101：1 CPU、1 GiB memory
  ├── Pod UID p-102：2 CPU、2 GiB memory
  └── Pod UID p-103：1 CPU、4 GiB memory
```

直覺上，我們可能會說：「YuniKorn 正在排這三個 Pod。」這句話不算錯，但還不夠精確。YuniKorn 必須同時知道：這些 Pod 是否屬於同一份工作、每一個 Pod 要求多少資源、哪些需求還在等待，以及哪些需求已經被放到某個 node。

> **先記住這一句**
>
> **YuniKorn 分配的是「被宣告的資源需求」，不是 CPU 使用率本身。**

先建立最小模型：

名稱	在本指南中的意思
application	scheduler 對一組相關工作的邏輯分組；不是 Kubernetes API object
Pod / task	standard shim 中，一個 Pod 對應一個 shim task
pending request / ask	還在等待 node placement 的一筆資源需求
allocation	core 已把該需求放到一個 node，並更新 scheduler accounting
node	可承載 allocation 的 Kubernetes Node 之 scheduler-side 模型
resource vector	例如 `{cpu: 2, memory: 2 GiB}` 的多維需求

同一個 application 可以包含多個 Pod。standard shim 使用 Pod UID 作為 task ID，也用它當 allocation key。因此，上例會有一個 application identity，以及三個互不相同的 allocation key。

```
application ID：analytics-job

allocation key p-101 ── pending 或 allocated
allocation key p-102 ── pending 或 allocated
allocation key p-103 ── pending 或 allocated
```

`ask` 在 core source 中仍是常見用語，但不要把它教成 YuniKorn 1.9.0 scheduler-interface 裡另一種獨立的 current message。1.9.0 的 interface 傳送 `Allocation`；在 core 的普通路徑中，同一個 allocation object 會從 pending 狀態被標成 allocated，allocation key 保持不變。

這也說明了兩組不能混在一起的關係：

```
application != Pod
pending demand != allocated placement
```

resource request 是 accounting fact。它表示 workload 宣告「若要放置這個 Pod，請替我計入這些資源」。它不表示 process 此刻真的用了多少 CPU，也不表示 memory resident set size。實際使用率可以遠低於或高低波動；scheduler 的 placement 判斷仍以 request 與 allocatable capacity 等資料為基礎。

YuniKorn 在這一層能建立的事實：哪個 application 提出哪一筆需求、需求包含哪些 resource dimensions，以及這筆需求目前是 pending 還是已被 scheduler 配置到 node。

不能由此推論：node 已經存在足夠的實體空間、Pod 已 bind、container 已啟動，或 application 已有實際進展。

> **Checkpoint**
>
> 三個 Pod 共用同一個 application ID，但 UID 不同。若其中一個 allocation 已成立，另外兩個仍 pending，哪些 identity 是共享的？哪些 state 必須分開追蹤？resource request 又能證明到哪裡？

> **Deep Dive（選讀）**
>
> - scheduler-interface 1.9.0 將 pending 與 allocated 資訊放在 `Allocation` message 的方式
> - extended resources 與多維 resource vector
> - Pod overhead 如何進入 request 計算
> - foreign allocation：不是由 YuniKorn 排出的 Pod 如何進入 node accounting

第二章：從 Pod 出現到 YuniKorn 做出 allocation，中間發生什麼？

假設使用者提交一個 Pod。稍後，YuniKorn 選出 `node-a`。這是否表示 Pod 已經開始執行？

不表示。從 API request 到真正的 process，中間有多個 authority boundary：

```
Kubernetes Admission
        ↓
API server 中存在 Pod object
        ↓
YuniKorn application registration / queue placement
        ↓
pending resource demand
        ↓
core allocation：選 node、更新 accounting
        ↓
shim 執行 bind
        ↓
kubelet admission / container runtime execution
```

> 先記住這一句
>
> **Allocation 不等於 Pod 已經在跑。**

`Admission` 在 Kubernetes 有明確含義：API request 在被持久化前，經過 admission plugins 或 webhooks 的 mutation 與 validation。為避免混淆，本指南不把 YuniKorn application acceptance 簡稱為 admission。

當我們說「YuniKorn 接受 application registration」時，安全的意思是：core 找到 partition、完成 application placement、把 application 登記到 leaf queue，並回覆 shim 該 registration 已被接受。它不表示 application 已有 pending request，更不表示資源可用。

接著，shim 把 Pod 的 UID、resource request、priority 與 constraint 翻譯成 core 能處理的需求。core 的責任是依 queue policy 與 node feasibility 做 scheduling decision，並維護 application、queue 與 node 的 accounting。

在普通的新 allocation 路徑中，core 成功 allocation 可以支持以下結論：

- 它在該次 scheduling decision 中選出一個 node。
- 需求通過當時的 queue、user 與 node resource 檢查。
- shim 提供的 predicate check 接受該 placement。
- core 已把該需求計入所選 node 與 queue。
- allocation response 已把 node decision 傳回 shim。

但接下來仍可能失敗。shim 還要處理 volume binding，並向 Kubernetes API 發出 Pod bind。即使 bind 成功，kubelet 仍要在 node 上接納 Pod，container runtime 仍要建立 sandbox、取得 image 並啟動 container。

例如：

```
core 選出 node-a
        ↓ success
shim 綁定 volume
        ↓ failure
Pod 沒有開始執行
```

或：

```
Pod bind 到 node-a
        ↓
kubelet 無法啟動 container
        ↓
allocation 曾成立，但 workload 沒有真正開始工作
```

各 component 能證明的範圍不同：

Component	它擁有的主要事實	它不能單獨證明的事
scheduler core	scheduling、queue policy、allocation accounting	API bind 或 process execution
shim	Pod / task 翻譯、callback、Kubernetes API 操作	container 實際在 node 上運作
API server	Kubernetes object 與 bind state	process 已啟動或退出
kubelet / runtime	node-local admission 與 container execution	core accounting 已收斂

shim source 中的 `Allocated`、`Bound` 等 task state 有助於追 code，但它們是 implementation detail，不是初學者需要先背的公開契約。

YuniKorn allocation 的 bounded guarantee：core 已做出並記錄 node placement decision。

它不保證：bind 成功、kubelet 接納、container 進入 Running、application 完成初始化，或 workload 產生有用輸出。

> **Checkpoint**
>
> 監控顯示 core 已為 Pod 選到 node。此刻你能確定哪些 scheduler facts？若使用者說「YuniKorn 說成功，但程式沒有跑」，接下來應分別向 shim、API server 與 kubelet 查什麼？

> **Deep Dive（選讀）**
>
> - shim task FSM 中的 `Allocated` 與 `Bound`
> - scheduler cache 的 `AssumePod`
> - volume binding 為什麼位於 core allocation 之後
> - bind failure 如何回到 allocation release

第三章：多個 workload 怎麼共享有限資源？

現在有兩個團隊共用同一個 cluster：

```
root
  ├── queue-a：Team A 的 applications
  └── queue-b：Team B 的 applications
```

Queue A 與 Queue B 都想使用相同的 CPU 和 memory。最直接的做法是把一半機器永久劃給 A、另一半永久劃給 B，但這會讓閒置容量無法被利用。另一個極端是完全先搶先贏，這又可能讓某個團隊長期拿不到資源。

> **先記住這一句**
>
> **Guaranteed resource 不等於 reserved hardware。**

YuniKorn 用 hierarchical queues 表達資源共享政策。application 被放在 leaf queue；parent queue 的 usage 是所有 descendants usage 的聚合。hierarchy 讓 policy 可以同時表達「整個部門」與「部門內各團隊」的共享關係，但它不是實體拓撲，也不表示某個 queue 擁有特定 node。

先分清兩個最重要的數字：

概念	它建立的界線	它不是什麼
maximum resource	queue subtree 同時可持有 allocation 的 hard cap	capacity 一定存在的承諾
guaranteed resource	fair ordering 與 ordinary preemption 使用的 policy baseline	預留機器、SLA、完成期限

假設 cluster 有 100 CPU：

```
queue-a guarantee = 40，max = 80，目前使用 20
queue-b guarantee = 40，max = 80，目前使用 70
```

Queue A 沒有把自己的 40 CPU 變成一組閒置硬體。當 A 只有 20 CPU 的需求時，B 可以在有可用 node 且未超過 max 的條件下使用更多容量。教學上可以把 B 超過 guarantee 的 30 CPU 稱為 borrowing，但系統裡沒有一筆具名的實體資源貸款，也沒有「這顆 CPU 屬於 A」的 ownership。

guarantee 會影響 fair queue sorting。當 queue 有 pending demand 時，core 比較它們相對於 guarantee 的 usage；相對 usage 較低者通常先被考慮。parent queue 固定使用 fair policy 排序 children。

如果 A 後來增加需求，A 的 guarantee 建立的是 policy position：A 低於 baseline，而 B 高於 baseline。這可讓 A 在 fair ordering 中獲得較有利位置，也可能在符合 priority、preemption fence、單一 node fit 等條件時，從高於 guarantee 的 queue 尋找 preemption victim。

但 guarantee 沒有創造 capacity。若所有 node 都不合適、總 guarantee 超過 cluster capacity、max hierarchy 擋住 placement，或 reclaim 流程失敗，A 仍可能沒有進展。

YuniKorn 保證的政策邊界：max 限制 allocation；guarantee 參與 fair-share 計算，並在 ordinary inter-queue preemption 中形成保護與 reclaim baseline。

不能由此推論：queue 已擁有一組硬體、低於 guarantee 就會立刻拿到資源，或每個 pending request 都會在有限時間內完成。

> **Checkpoint**
>
> Queue A 低於 guarantee，Queue B 高於 guarantee。這個狀態讓 A 在 policy 上得到什麼？若 A 的 Pod 仍放不進任何 node，guarantee 又有哪些事完全沒有證明？

> **Deep Dive（選讀）**
>
> - CPU 與 memory 同時存在時的 multidimensional fairness
> - child queue 的 effective guarantee / max 如何受 parent 約束
> - user / group headroom 與 queue headroom 的差異
> - priority 如何和 fair ordering 共同決定先後順序

第四章：為什麼「公平」的工作還是可能跑不起來？

考慮一個看似有 4 CPU 空間的 cluster：

```
node-a：剩 2 CPU
node-b：剩 2 CPU

pending Pod：需要 4 CPU
```

把整個 cluster 加總後，free CPU 的確是 4。但 Pod 不能各放一半到兩個 node；它需要某一個 node 同時提供 4 CPU。

> **先記住這一句**
>
> **Fairness 決定誰先被考慮；feasibility 決定它到底能不能放得下。**

這裡需要把四種問題分開：

問題	它問的是什麼？
accounting consistency	scheduler 記錄的 pending、allocated、node usage 是否彼此一致？
fairness	多個可競爭的 requests 中，誰先被考慮？
feasibility	request 是否能在某一個 node 上通過 resource 與其他 predicates？
liveness	隨著時間與事件前進，request 是否終究能取得進展？

fair ordering 可以正確地把這個 4 CPU request 排在最前面，但它無法把兩個 2 CPU 碎片合成一個可用 node。accounting 也可能完全正確：兩個 node 各剩 2 CPU，request 需要 4 CPU。沒有任何數字矛盾，但 request 還是不能 allocation。

這就是 YuniKorn 需要 reservation 的物理原因。若一個較大或較挑剔的 request 一直等不到完整空間，較小的 requests 可能每次都先吃掉剛釋放的碎片。經過等待後，YuniKorn 可以為該 request 在一個合格 node 上建立 reservation，之後優先嘗試它，並阻止不相干的 YuniKorn allocations 插入該 node。

```
reservation = 「這個 request 對這個 node 有優先權」

reservation != allocation
reservation != 已釋放的 CPU / memory
reservation != Pod bind
```

reservation 記錄 application、allocation key 與 node 的關係。它不會趕走現有 workload，也不會把 node 的 `availableResource` 假裝增加。只有既有 allocation 真正 release，或其他實際容量出現後，reserved request 才可能 allocation。

reservation 是 progress mechanism：它降低大型 request 被後來的小 request 持續插隊的風險。它不是 completion theorem。reservation 仍可能因 request、application 或 node 消失而被清除，也可能受 headroom、predicate、timeout 或其他狀態變化影響。

公平能保證的有限範圍：按照 policy 決定考慮順序。

公平與 reservation 都不能保證：一定存在 feasible node，或 workload 會在有限時間內完成。

> **Checkpoint**
>
> 一筆 request 已被公平地排在最前面，而且有 reservation，卻仍然 pending。請列出 reservation 已建立的 scheduler state，以及仍未成立的 physical facts。為什麼這不代表 fairness 壞掉？

> **Deep Dive（選讀）**
>
> - reservation 在 allocation、ask removal 與 timeout 時的 cancellation
> - resource fragmentation 與單一 node feasibility
> - required-node request 為什麼需要不同處理
> - reservation、unschedulable signal 與 autoscaling 的互動邊界

第五章：Preemption 什麼時候才真的「拿回資源」？

回到 Queue A 與 Queue B。B 目前使用了超過 guarantee 的容量，A 低於 guarantee，且 A 有一筆高 priority request 找不到空間。YuniKorn 可能判斷：某些已存在的 allocations 必須讓出資源。

最危險的直覺是：「scheduler 選好 victim，資源就已經空出來。」實際上，中間是一條跨越 core、shim、API server、informer 與 kubelet/runtime 的流程。

> 先記住這一句
>
> **Preemption decision 不等於 capacity 已經被釋放。**

先看正常成功路徑：

```
core 選 victim
  ↓
victim 標成 preempted
  ↓
victim queue 增加 preemptingResource
  ↓
core 發出 release request，為 preemptor reserve node
  ↓
shim 嘗試刪除 victim Pod
  ↓
Kubernetes object 出現 terminal / deletion change
  ↓
informer 觀察到 change，shim 完成 task
  ↓
shim 把 release feedback 送回 core
  ↓
core 移除 allocation，更新 node / queue accounting
  ↓
scheduler model 中的 capacity 可再次使用
  ↓
preemptor 才可能 allocation、bind、執行
```

`marked preempted` 只證明 core 已選定這個 victim，並避免它再次被當成新的 victim。`preemptingResource` 表示預計回收中的 accounting intent。兩者都沒有從 node 的 allocated resources 中移除 victim。

因此必須保持四個界線：

```
marked preempted != allocation released
delete requested != Pod absent
Pod absent from API != universal proof that process exited
core allocation released != universal proof of physical process exit
```

對 core 而言，capacity 真正可再次排程的時刻，是 core 收到並處理 release feedback、從 node 與 queue accounting 移除 victim allocation。對實體 process 而言，只有 kubelet / runtime 擁有 node-local termination truth。尤其在 force deletion 下，API server 可以先移除 Pod object，而不等待 kubelet 確認 process 已退出；若 node 當時不可達或與 control plane 斷線，runtime termination 仍無法確認，舊 process 可能暫時存在。

一個有邊界的 case study：YUNIKORN-3382

YUNIKORN-3382 指出 YuniKorn 1.9.0 shim 的一條特定失敗路徑。core 要求釋放 preemption victim 後，shim 呼叫 `DeleteTaskPod()`；若 delete 回傳 error，該 handler 只記錄 error，沒有在這條 traced path 中 retry、re-drive，或把失敗回報給 core。

```
正常路徑：
delete 成功
  → informer / task completion
  → release feedback
  → accounting convergence

#3382 路徑：
delete 失敗
  → error 被記錄後丟下
  → victim 仍 allocated 且 marked preempted
  → core 沒收到 release feedback
  → fixed-capacity 情境下，preemptor 持續 pending
```

這個狀態可以是 internally consistent：core 仍把實際存在的 victim 算在 allocated resources 裡，node 沒有憑空增加 capacity；同時，preemption intent 也仍被記錄。問題不是帳面數字必然互相矛盾，而是沒有 component 繼續擁有「把失敗 delete 對帳完成」的 obligation，造成 liveness failure。

這不應被寫成所有未來事件下的永久 deadlock。victim 之後自行完成、外部成功刪除、application / node 被移除、新 capacity 出現，或 restart reconstruction，都可能改變結果。#3382 能支持的是：**在原 issue 的 fixed-capacity、沒有其他解鎖事件的情境中，progress 會停住。**

也不要沿用早期對 `preemptingResource` 方向的錯誤描述。stable 1.9.0 formulas 會讓 stale preempting resource 降低算出的 preemptable resource，並提高算出的 remaining guarantee。這個 policy distortion 值得在 source review 中注意，但 beginner main path 只需掌握：它不是已經釋放的 node capacity。

preemption policy decision 能保證的事：core 找到符合當時規則的 victims，標記 reclaim intent，並要求 shim 採取動作。

它不能單獨保證：delete 成功、informer feedback 抵達、core accounting 收斂，或 process 已退出。

> **Checkpoint**
>
> 請把同一次 preemption 分成四個時刻：policy decision、Kubernetes action、scheduler resource release、physical process exit。每個時刻由哪個 component 提供 evidence？如果只看到 `preempted=true`，還不能推論什麼？

> **Deep Dive（選讀）**
>
> - YUNIKORN-3382 的 stable source audit 與缺少的 regression test
> - victim-selection、remaining guarantee 與 preemptable resource formulas
> - 為 retry 加上 Pod UID precondition，避免刪除同名 replacement Pod
> - retry ownership、backoff、cancellation 與 application lifecycle
> - Kubernetes force deletion 為什麼不能直接證明 process exit

第六章：從 Pending Pod 追到第一次 YuniKorn Contribution

到這裡，你已經具備開始讀 source 的模型。Contribution Ready 不表示要先讀完整個 scheduler core；它表示你能從一個公開症狀，沿著 identity、authority 與 state transition，建立一條可檢查的 evidence chain。

YuniKorn core 與 Kubernetes shim 都是 Go code。你需要能追 function call、struct field、test setup 與 callback，但不必先學完整套 Go 語法或整個 Kubernetes scheduler framework。

> 先記住這一句
>
> **Contribution Ready 是能追出一條 evidence chain，不是讀完整個 scheduler。**

本章只走一條主路徑：

> **Pending demand → core allocation**

假設公開症狀是：「Pod 一直 Pending，但 queue 看起來還有空間。」不要立刻搜尋所有 scheduler code，也不要先猜是 fairness bug。依序建立 evidence：

1. 固定版本與現象

先確認報告對應 YuniKorn 1.9.0 standard scheduler mode。記錄 Pod UID、application ID、resolved queue path、resource request，以及 Pod 是否已有 `spec.nodeName`。

同名 Pod 可能已被重建；名字不足以連接 core request。standard shim 中，Pod UID 才是 task ID 與 allocation key。

2. 先判斷 authority boundary

```
Pod 尚未 bind
  ├── application registration 是否成功？
  ├── allocation key 是否成為 pending request？
  ├── queue / user headroom 是否允許？
  ├── 是否存在單一 feasible node？
  └── core 是否曾回傳 allocation？
```

若 core 從未 allocation，先留在 scheduling path。若 core 已 allocation，問題已跨到 shim bind 或 kubelet/runtime boundary。這一步能避免在錯誤的 repository 修錯問題。

3. 從 focused test 建立最小正常模型

從 core 的兩個小測試開始：

- `TestAllocateDeallocate`：觀察同一筆 request 如何在 pending 與 allocated accounting 間轉換。
- `TestTryAllocateFit`：建立一個 application、一筆 request 與一個 node，確認成功 placement 回傳哪個 node。

test 能保護的是一個 bounded case。它不能自動證明 production symptom 有相同原因，但能告訴你最小正常 transition 應該長什麼樣子。

4. 只讀 owning transition

這筆 core request 不是憑空產生。standard shim 從 Kubernetes Pod 建立 task 與相關 metadata，再由 `Task.updateAllocation()` 組成 scheduler-interface allocation request 並送往 core；core 再依 application ID 把 request 交給對應的 application。

接著沿著很短的 source path：

```
Application.AddAllocationAsk
        ↓
Application.tryAllocate
        ↓
Application.tryNode
        ↓
Node.TryAddAllocation
        ↓
PartitionContext.allocate
```

你要追的不是每一行，而是四個 state change：

1. pending resource 在哪裡增加？
2. node fit 與 predicate 在哪裡判斷？
3. request 何時被標成 allocated？
4. application、queue 與 node accounting 何時一起更新？

5. 寫出 deterministic failure timeline

把 symptom 改寫成可反駁的序列：

```
Pod UID p-101 被 shim 觀察
  → application 已註冊到 root.team-a
  → allocation key p-101 已進入 pending
  → fair ordering 選到該 application
  → 每個 candidate node 都在某個明確檢查失敗
  → 因此沒有 core allocation response
```

如果你找不到「哪個明確檢查失敗」，這就是下一個 test 應縮小的範圍。先讓 focused test 重現那個 transition，再討論 minimal fix。

6. 保持 evidence discipline

Evidence	可以支持	不能單獨支持
official docs	intended public behavior	每個 internal call order
pinned source	1.9.0 observed implementation	永久公開契約
focused test	一個 bounded scenario 的預期	所有 production interleaving
issue report / comment	symptom、reproduction、hypothesis	已被專案承諾的 semantics
suspicious code	值得測試的 failure hypothesis	root cause 已證明

一個適合首次 contribution 的成果，可能只是：更小的 reproduction、能先紅後綠的 focused test、精確的 failure timeline，或把錯誤 evidence boundary 寫清楚。這些都比沒有 test 的大範圍重構更接近可 review 的改動。

這條 source bridge 能讓你建立的能力：從 Pod identity 追到 pending request，再追到 allocation 的 owning transition 與 focused tests。

它不要求：讀完整個 queue hierarchy、所有 preemption algorithms、gang scheduling 或 Kubernetes runtime internals。

> **Checkpoint**
>
> 面對一個 Pending Pod，在修改 code 前，你會如何固定版本、確認 Pod / application / allocation identity、判斷問題屬於 core 還是 shim、選一個 focused test，並把「可疑 source」提升成可重現的 root-cause evidence？

> **Deep Dive（選讀）**
>
> - queue fairness path：`TestSortQueuesWithGuarantees` → `sorters.go` → usage-ratio comparison
> - preemption / release path：core `TestPreemption` → shim release callback → informer completion → core removal
> - fault injection：bind failure、delete failure與 node event 的 bounded oracles
> - #3382 regression opportunity：failed delete 的 re-drive、UID safety 與 lifecycle cancellation

技術依據與延伸閱讀

以下 YuniKorn 文件在本原型撰寫時對應 1.9.0。implementation evidence 固定到指定 core / shim commit；未來 stable 文件若更新，不應反向改寫 1.9.0 的 source facts。

第一章

- YuniKorn Workloads Overview
- Labels and Annotations in YuniKorn
- scheduler-interface `si.proto` — 1.9.0-0
- core `allocation.go` — pinned 1.9.0 source
- shim `metadata.go` — Pod UID/task identity

第二章

- YuniKorn Architecture
- core application registration handling
- shim allocation callback
- shim task binding path
- Kubernetes Pod Lifecycle
- Kubernetes Kubelet Sync Loop

第三章

- Partition and Queue Configuration — resource semantics
- Sorting Policies
- Preemption Design
- core queue sorter
- `TestSortQueuesWithGuarantees`

第四章

- YuniKorn Resource Reservation
- core reservation object
- core node reservation handling
- `TestBasicReservation` and `TestReservationForTwoQueues`
- Preemption tests for victims split across nodes

第五章

- YuniKorn Preemption Design
- core preemption transition
- core allocation release/accounting convergence
- shim core-initiated release handler
- shim informer completion path
- `TestPreemption`
- Kubernetes Forced Pod Termination

第六章

- `TestAllocateDeallocate`
- `TestTryAllocateFit`
- core application allocation path
- core node allocation path
- core partition allocation commit

Case studies：不是 YuniKorn 1.9.0 公開契約

- YUNIKORN-3382：failed victim Pod delete 的 missing re-drive；stable source 支持 bounded failure chain，但 issue 仍 open
- YUNIKORN-3386：preemption fault-injection scenario；屬於 red-by-design issue evidence，不是 stable regression contract

方法來源與查證聲明

本指南的第一原理教學方法，源自 Yen-Hua Chen 在 *Streaming System + Compass* 中記錄的工程推理：先辨認 physical actors、state transitions、authority boundaries、evidence boundaries 與 failure timeline，再討論抽象與 API。

- **Streaming System + Compass — Yen-Hua Chen**
- https://github.com/hikaru-212/streaming-system-compass
- Documentation license: CC BY 4.0

Compass architecture 並未直接套用到 YuniKorn。所有 YuniKorn-specific claims 都已獨立對照 YuniKorn 1.9.0 文件、pinned stable source 與 focused stable tests；物理機制不相符的類比則明確拒絕：

- queue guarantee != reserved hardware
- allocation != running Pod
- scheduler accounting != runtime truth
- restart reconstruction != durable workflow replay

作者備註

這份 short main path 刻意省略完整 scheduler-interface schema evolution、所有 application / task FSM states、完整 preemption formulas、reservation cancellation edge cases、HA / recovery races、gang scheduling、autoscaler internals，以及 user / group limit 的特殊情況。

未來 Expanded / Deep Dive 版可再處理 multidimensional fairness、required-node preemption、recovery reconstruction、gang placeholders、fault-injection harness，以及從 #3382 regression test 到 production retry design 的完整貢獻流程。